Authentication and Authorization in Django

Authentication, Authorization, Permissions

Simone Ricci

Authentication and Authorization

Authentication and Authorization

Authentication: the process of verifying the **identity** of a user. It could be achieved with a set of credentials, such as a username and password.

Authorization: the process of determining whether a user has **permission** to access a specific resource or perform a specific action.

Authorization is typically implemented by defining access control policies that determine which users or groups of users are allowed to perform specific actions or access specific resources.

Authentication and authorization work together to secure web applications by ensuring that only authorized users have access to sensitive data and functionality.

In e-commerce website :

- **Authentication**: would ensure that **only registered users** with valid credentials can access their account information
- **Authorization**: would **limit what actions** each user can perform, such as preventing a regular user from accessing the administrative dashboard.

Authentication	Authorization
The **identity** of users are checked for providing the access to the system.	The user's **authorities** are validated for accessing the resources.
Before the authorization process.	After the authentication process.
Needs **user's login details**.	Needs the **user's privilege** or security levels.
Popular Authentication Techniques: ● Password-Based Authentication ● 2FA/MFA ● Single sign-on (SSO) ● Social authentication	Popular Authorization Techniques: ● Role-Based Access Controls ● JSON web token (JWT) ● OpenID Authorization ● OAuth 2.0 Authorization
Credentials **can be changed** when required by the user.	The authorization permissions **cannot be changed by user** as these are granted by the owner of the system
It is **visible** at user end.	It is **not visible** at the user end.

Authorization and authentication protocols

OAuth (Open Authorization): is an **open standard protocol** used for authorization and authentication, allowing users to grant third-party applications access to their protected resources without sharing their login credentials.

OpenID: This is an identity layer built on top of the OAuth 2.0 protocol that provides user authentication.
It allows for the exchange of identity information between parties and allows users to authenticate to multiple applications using a single set of credentials.

LDAP: Lightweight Directory Access Protocol (LDAP) is a protocol for accessing and managing directory services.
It is commonly used for centralized authentication and authorization in enterprise environments, where users and permissions are managed through a central directory.

SAML (Security Assertion Markup Language): This is an XML-based protocol used for exchanging authentication and authorization data between parties.
It is commonly used in enterprise environments to provide single sign-on (SSO) capabilities.

OAuth

OAuth is designed to let users grant limited access to their resources on one site (the “resource server”) to another application (the “client”), without handing over their username and password.

It does this by issuing tokens—short-lived credentials scoped to specific actions—that the client can use to act on the user’s behalf.

1. **User Initiates Login**: “Connect with Google” in the client app.
2. **Redirect to Authorization Server**: the client sends you to Google’s OAuth endpoint, including: client_id, redirect_uri (where the token/code should be sent), a scope (what permissions it wants, e.g., calendar.read)
3. **User Grants Consent**: Google prompts: “App X wants to view your calendar. Allow?”
4. **Authorization Code Issued**: After consent, Google redirects you back to the client’s redirect_uri with a short-lived authorization code.
5. **Client Exchanges Code for Token**: The client’s backend talks directly to Google, sending the code plus its client_secret to receive an access token (and optionally a refresh token).
6. **Accessing the Resource**: The client uses the access token to call Google’s Calendar API and fetch events.

LDAP

LDAP (Lightweight Directory Access Protocol) is a standard network protocol used for accessing and maintaining distributed directory information services over **an IP network.**

Unlike OAuth or OpenID Connect—both web-centric protocols built around HTTP and tokens—LDAP is geared toward querying and modifying a centralized, hierarchical directory.

1. **Client Connection**: an application (e.g. mail server, single-sign-on gateway) opens a TCP connection to port 389 (or 636 for LDAPS).

2. **Bind Request**: the client sends a Bind request with a DN (Distinguished Name) (e.g. id=alice, ou=People, dc=example, dc=com) and password.

3. **Bind Response**: The LDAP server verifies credentials against its directory and replies with success or an error code.

4. **Search (Optional)**: on successful Bind, the client can perform Search operations to retrieve attributes (e.g. user groups, policies).

5. **Unbind**: Client closes the session.

Web application security

Web applications are often targeted by attackers seeking to exploit vulnerabilities and gain unauthorized access to sensitive data or resources.

Several common security risks:

- **SQL Injection**: target the application's database layer. Inject malicious SQL statements into an application's database query.

  Potentially allowing **bypass authentication, access sensitive data, or modify data in the database**.

- **Cross-site scripting (XSS) attacks**: XSS attacks occur when attackers inject malicious code into a web page viewed by other users.

  It can be used to steal user credentials, redirect users to malicious websites, or perform other unauthorized actions.

- **Cross-site request forgery (CSRF) attacks**: an attacker sends a **forged request from a user's browser** that appears to be legitimate.

  It allows the attacker to execute unauthorized actions on behalf of the user.

SQL Injection

An attacker can submit the following form fields:

$username: **' OR '1'='1**

$password: (anything, e.g. empty string)

The resulting query becomes:

SELECT * FROM users WHERE username = ' ' OR '1'='1' AND password = ' '

Because the condition ' ' OR '1'='1' is always true, every row in the users table satisfies it.

mysqli_num_rows returns ≥1, and the attacker is logged in, often as the first user in the table (e.g. an admin).

```php
<?php
// login.php

// Grab credentials from POST
$username = $_POST['username'];
$password = $_POST['password'];

// Build query - **vulnerable** to injection!
$sql = "
  SELECT *
  FROM users
  WHERE username = '$username'
    AND password = '$password'
";

// Execute the query
$result = mysqli_query($conn, $sql);
if (mysqli_num_rows($result) === 1) {
    // Success: start session, redirect, etc.
} else {
    echo "Invalid credentials.";
}
?>
```

Cross-site scripting (XSS)

An attacker injects malicious client-side scripts (usually **JavaScript**) into pages viewed by other users. When those pages render in a victim's browser, the injected code executes in the security context of the site.

```php
<?php
// search.php
$term = $_GET['q'];
// Danger: directly embedding user input in HTML
echo "<h1>Search Results for “{$term}”</h1>";
// … display results …
?>
```

Reflected XSS: The malicious script comes from the current HTTP request (e.g. query string, form field) and is "reflected" back in the response.

```
https://example.com/search.php?q=<script>alert('XSS')</script>
```

Stored (Persistent) XSS: The attacker's payload is stored on the server (e.g. in a database, comment field) and served to all users who view that content.

```
<h1>Search Results for “<script>alert('XSS')</script>”</h1>
```

DOM-Based XSS: The vulnerability arises entirely in client-side code: JavaScript reads from an unsafe source (e.g. location.hash) and writes HTML without sanitization.

Cross-Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) is a web-application attack that tricks an authenticated user's browser into **making unwanted requests** to a site where they're logged in—thereby performing actions on their behalf without their consent.

1. **Victim Authentication**: The user logs into **bank.example.com** and receives a **session cookie**.
2. **Attacker and Victim**: The victim visits **attacker.com** (e.g. by clicking a link or viewing an image).
3. **Malicious Request**: The attacker's page contains **hidden code** (an HTML form, image tag, or AJAX request) targeting **bank.example.com**, for example:

   <img src="https://bank.example.com/transfer?amount=1000&to=attackerAccount" style="display:none"> The browser automatically includes the user's valid session cookie for bank.example.com.
4. **Bank Processes Request**: bank.example.com sees an authenticated request to transfer funds—and, seeing the valid cookie, executes it—transferring money without the user's approval.

Django authentication and authorization system

Django offers its built-in authentication and authorization system.

They are **integrated with other parts of the framework**, such as the Admin interface and Forms framework.

The authentication system uses a combination of authentication backends and authentication middleware to **verify a user's identity**.

- **Authentication backends**: are responsible for checking the validity of user credentials (such as username and password).
- **Authentication middleware**: is responsible for managing user sessions, handling authentication requests, and enforcing authentication policies.

It **can also be extended** by defining custom authentication backends or using third-party packages, such as OAuth or LDAP.

Authentication in Django

User class

Django provides both authentication and authorization together.

The base class **User** is the core of the **authentication system**.

Each instances of this class typically represent one user-client.

Only one class of user exists in Django's authentication framework!

'superusers' or **'staff'** users are just user objects **with special attributes set**, not different classes of user objects.

The primary attributes of the default user are:

- **username**: Required
- **password**: Required. Hash and metadata of the password. (Django doesn't store the raw password.)
- **email**
- **first_name**
- **last_name**

Custom User

The **User** model is a subclass of Django's **AbstractBaseUser** or directly from **AbstractUser** class, which is a high-level base class for custom user models. (must change **AUTH_USER_MODEL** in **setting.py**)

Custom user model are defined by subclassing the AbstractBaseUser class and **overriding** its default behavior.

- Keeping all user related information in one model to removes the need for additional or more complex database queries.

It may be more suitable to store app-specific user information in a model that has a relation with user model.

- That **allows each app to specify its own user data requirements** without potentially conflicting or breaking assumptions by other apps!

Keep your user model as simple as possible, focused on authentication, and following the minimum requirements Django expects custom user models to meet!

```
class CustomUser(AbstractUser):
    age = models.PositiveIntegerField(null=True, blank=True)
```

Custom User manager

If your user model defines the same field as Django's default user, you can use the standard **UserManager.**

When your user model defines different fields, you'll need to define a **custom manager** that extends **BaseUserManager**.

A manager in Django is a class that provides a way to query the database and retrieve model instances.

```
class CustomUserManager(BaseUserManager):
    def create_user(self, email, password=None, **extra_fields):
        if not email:
            raise ValueError('The Email field must be set')
        email = self.normalize_email(email)
        user = self.model(email=email, **extra_fields)
        user.set_password(password)
        user.save(using=self._db)
        return user

    def create_superuser(self, email, password=None, **extra_fields):
        extra_fields.setdefault('is_staff', True)
        extra_fields.setdefault('is_superuser', True)
        return self.create_user(email, password, **extra_fields)
```

```
from django.contrib.auth.models import AbstractBaseUser

class CustomUser(AbstractBaseUser):
    # ... fields and methods ...
    objects = CustomUserManager()
```

Override two methods:

- **create_user(args)**
- **create_superuser(args)**

Admin User changes

Since Admin User is tightly coupled to the default User model **we needed to adapt it**.

It is possible extending the existing **UserAdmin** class to use our new **CustomUser** model.

To control which fields are listed we use **list_display**.

To actually edit and add new custom fields (like age) we must also add **fieldsets** (for fields used in **editing** users data) and **add_fieldsets** (for fields used when **creating** a user).

```
class CustomUserAdmin(UserAdmin):
    add_form = CustomUserCreationForm
    form = CustomUserChangeForm
    model = CustomUser
    list_display = [
        "email",
        "username",
        "age",
        "is_staff",
    ]
    fieldsets = UserAdmin.fieldsets + ((None, {"fields": ("age",)}),)
    add_fieldsets = UserAdmin.add_fieldsets + ((None, {"fields": ("age",)}),)
```

User creation

The most direct way to create users object is to use the included **create_user()** helper function.

```
>>> from django.contrib.auth.models import User
>>> user = User.objects.create_user('john', 'lennon@thebeatles.com', 'johnpassword')
```

The **admin page** provides a convenient way to view and manage users, groups, and permissions.

Users can be created and deleted like any other Django model.

Give the ability of user to register by themself through a specified **sign-up page**.

This can be obtained by presenting a form to the user to insert data **that will populate a new User object.**

We will see how to makes login, logout and sign-up an user in the next slides

Login

Django provides a default view for a login page via class **LoginView**:

- If called via **GET**, it displays a login form that POSTs to the same URL of this view.
- If called via **POST** with user submitted credentials, it tries to log the user in.
  - If login is successful, the view redirects to the URL specified in **next**. If next isn't provided, it redirects to **settings.LOGIN_REDIRECT_URL** (which defaults to /accounts/profile/).
  - If login isn't successful, it redisplays the login form.

This class looks for login template (**registration/login.html**) by default. This template gets passed context variables:

- **form**: A Form object representing the AuthenticationForm.
- **next**: The URL to redirect to after successful login.

LOGIN_REDIRECT_URL setting can be change in the **settings.py.**

```
class MyLoginView(LoginView):
    form_class = CustomAuthenticationForm
    template_name = "registration/login.html"
```

AuthenticationForm class

It is a **built-in Form class** in Django that provides a standard way to authenticate users.

It is typically used in conjunction with Django's built-in authentication views and authentication system.

AuthenticationForm provides the fields and validation logic for this form.

It includes two required fields: **username and password**.

AuthenticationForm validates the user's input by checking that the username and password match a user in the system.

By default, AuthenticationForm rejects users whose **is_active** flag is set to False.

To change this behavior with a custom policy, just override the **confirm_login_allowed()** method.

This method should raise a **ValidationError** if the given user may not log in.

Login in a view

Use **authenticate()** to verify a set of credentials.

It **takes credentials as keyword arguments**, username and password for the default case, checks them against each authentication backend, and **returns a User** object if the credentials are valid for a backend.

If the credentials aren't valid for any backend or if a backend raises **PermissionDenied**, it returns **None**.

An authenticated user can be attached to the current session with **login()** function.

```
from django.contrib.auth import authenticate, login


def my_view(request):
    username = request.POST["username"]
    password = request.POST["password"]
    user = authenticate(request, username=username, password=password)
    if user is not None:
        login(request, user)
        # Redirect to a success page.
        ...
    else:
        # Return an 'invalid login' error message.
        ...
```

Logout

It is possible to check if an user is logged in checking the **models.User** attribute **is_authenticated**.

This is useful to perform action or display element based on this condition.

Thanks to the Django auth system, logout is really simple to achieve.

To log out a user who has been logged in via django.contrib.auth.**login()**, use django.contrib.auth.**logout()** within a view.

When you call **logout()**, the session data for the current request is completely cleaned out. **All existing data is removed.**

This is to prevent another person from using the same web browser to log in and have access to the previous user's session data.

Specify where to redirect a user upon logout, setting the variable **LOGOUT_REDIRECT_URL**

```
<!-- templates/base.html-->
...
{% if user.is_authenticated %}
  <p>Hi {{ user.username }}!</p>
  <p><a href="{% url 'logout' %}">Log out</a></p>
{% else %}
...
```

SignUp

The signup process in web development refers to the series of steps a user takes to create a **new account** on a website or web application.

SignUp process steps:

1. **Registration Form**: The user is presented with a registration form that asks for their personal information such as **name, email, username, and password**. The form may also ask for additional information.
2. **Validation**: The user's input is validated to ensure that it meets the requirements set by web application.
3. **Email Verification (optional)**: an email with a verification link to verify their email address is send to the new account.
4. **Account Creation**: the account is created and they are logged in to the website or web application.

SignUp Django

Django provides a **UserCreationForm** class, to make things easier. By default it comes with three fields: username, password1, and password2.

This class verifies that password1 and password2 match, validates the password using **validate_password()**, and sets the user's password using set_password().

It is possible to modify the creation form creating a new class that extends **UserCreationForm** class.

```
from django.contrib.auth.forms import UserCreationForm
from .models import CustomUser


class CustomUserCreationForm(UserCreationForm):
    class Meta:
        model = CustomUser
        fields = (
            "username",
            "email",
            "age",
        )  # new
```

The class definition of the new form could be pleased in a new module **forms.py** created in the relative app folder.

SignUp View

A simple way to create a view for signup is to extend the generic class-based view **CreateView** in our **SignUpView** class.

We specify the use of the CustomUserCreationForm and the template at registration/signup.html.

```
from django.urls import reverse_lazy
from django.views.generic import CreateView

from .forms import CustomUserCreationForm


class SignUpView(CreateView):
    form_class = CustomUserCreationForm
    success_url = reverse_lazy("login")
    template_name = "registration/signup.html"
```

The function **reverse_lazy()** redirects the user to the login page upon successful registration.

The reason of the *lazy* version is that for all generic class-based views the URLs are not loaded when the file is imported.

Session

All communication between web browsers and servers is via HTTP, which is **stateless**.

The messages between the client and server are completely independent of each other — there is **no notion of "sequence"**.

Sessions are the mechanism used by Django (and most of the others) for keeping track of the "**state**" between the site and a particular browser.

Sessions allow you to store arbitrary data per browser, and have this data available to the site whenever the browser connects.

Individual data items associated with the session are then referenced by a "**key**", which is used both to store and retrieve the data.

Django **uses a cookie containing a special session id** to identify each browser and its associated session with the site.

The actual **session data is stored in the site database** by default.

Django can be configured to store session data on your filesystem or in your cache

Using sessions

To use database-backed session, you need to add '**django.contrib.sessions**' to your **INSTALLED_APPS.**

When **SessionMiddleware** is activated, each **HttpRequest** object will have a **session** attribute, which is a dictionary-like object.

You can read it and write to **request.session** at any point in your view**.** You can access the **session** attribute within a view from the **request** parameter.

This session attribute represents the connection to the current user (current browser).

By default, Django only saves to the session database and sends the session cookie to the client when the session has been modified (assigned) or deleted.

```
# Get a session value by its key (e.g. 'my_car'), raising a KeyError if
present
my_car = request.session['my_car']

# Get a session value, setting a default if it is not present ('mini')
my_car = request.session.get('my_car', 'mini')

# Set a session value
request.session['my_car'] = 'mini'

# Delete a session value
del request.session['my_car']
```

Best practices for securing user authentication

Securing user authentication is crucial for protecting sensitive user data and preventing unauthorized access to an application.

- **HTTPS**: It encrypts communication between the server and the client. Django provides built-in support for HTTPS.
- **Strong passwords**: Django provides built-in support for password validation, and developers should use it to enforce password complexity requirements..
- **Multi-factor authentication**: an extra layer of security to user authentication by requiring users to provide two or more forms of authentication. Django provides third-party libraries for implementing multi-factor authentication.
- **Store passwords**: Passwords should never be stored in plain text. Django provides built-in support for password hashing and salting.
- **Rate limiting**: limits the number of authentication attempts that can be made within a certain time frame, preventing brute-force attacks.
- **Session timeouts**: log out users automatically after a certain period of inactivity.

Hashing and Salting

Password hashing involves taking a user's **plain-text password** and converting it into an **irreversible**, hashed value that is stored in a database.

Salting adds a **random string** to the user's plain-text password **before it is hashed**, which makes it much more difficult for an attacker to crack a user's password.

When a user creates an account or changes their password, Django generates a random salt value and adds it to the user's plain-text password.

It then hashes the salted password using the **default password hasher**, which is set to **PBKDF2** by default.

The resulting hashed value, along with the salt value, is then stored in the database.

When a user attempts to log in:

1. Django retrieves the stored salt value and uses it to salt the plain-text password entered by the user.
2. It then hashes the salted password and compares the result to the stored hash in the database.

Authorization in Django

Django authorization system

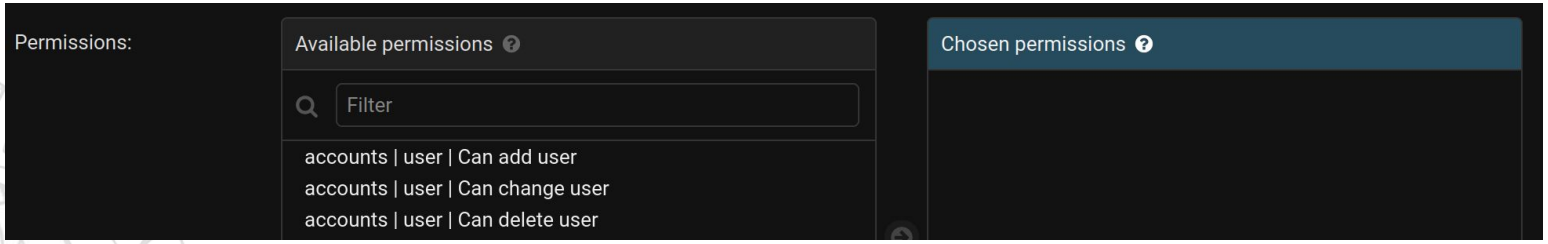
Once a user is authenticated, Django's authorization system comes into play.

This system is based on the concept of **permissions**, which define what actions a user is allowed to perform on a particular resource (URL) or object.

Django's permission system is **highly customizable and flexible**, allowing developers to define their own permissions or use one of the many built-in permissions provided by the framework.

Permissions can be defined at the **global level**, the **app level**, or the **model level,** and can be assigned to users or **groups of users**.

Groups are collections of permissions that can be assigned to users, making it easy to manage access control for different types of users or roles.



Permission

Permissions in Django are used to control access to models and their associated views.

They **define what a user can and cannot do** with specific models and their instances, such as view, add, change, or delete records.

Django provides a set of built-in permissions, including:

- **view**: Allows users to view records.
- **add**: Allows users to add new records. It is typically used to control access to views that create new instances of a model.
- **change**: Allows users to change existing records.
- **delete**: Allows users to delete records.
- **view_own**: Allows users to view their own records only.
- **change_own**: Allows users to change their own records only.

Permission

These built-in permissions can be used to control access to views in Django by checking if the user has the required permission before allowing them to perform the action.

Permissions are applied at the model level and **can be assigned to groups or individual users**.

By default, **all permissions** are assigned to the superuser group, which includes users with the **is_superuser** attribute set to **True**.

Defining permissions is done on the model class **Meta** section, using the **permissions** field.

You can specify as many permissions as you need in a tuple, each permission itself being defined in a **nested tuple** containing the permission name and permission display value.

```
class BookInstance(models.Model):
    # …
    class Meta:
        # …
        permissions = (("can_mark_returned", "Set book as returned"),)
```

Check permissions in Template

The current user's permissions are stored in a template variable called **{{ perms }}**.

You can check whether the current user has a particular permission using the specific variable name **within the associated Django "app"**:

- **{{perms.catalog.can_mark_returned }}** will be True if the user has this permission, and False otherwise.

It's possible to test for the permission using the template {% if %} tag as shown:

```
{% if perms.catalog.can_mark_returned %}
    <!-- We can mark a BookInstance as returned. -->
    <!-- Perhaps add code to link to a "book return" view here. -->
{% endif %}
```

Check permissions in View

Permissions can be tested in function view using the **@permission_required** decorator or in a class-based view using the **PermissionRequiredMixin**.

```
@permission_required('catalog.can_mark_returned')
@permission_required('catalog.can_edit')
def my_view(request):
    # …
```

function view

```
class MyView(PermissionRequiredMixin, View):
    permission_required = 'catalog.can_mark_returned'
    # Or multiple permissions
    permission_required = ('catalog.can_mark_returned', 'catalog.can_edit')
    # Note that 'catalog.can_edit' is just an example
    # the catalog application doesn't have such permission!
```

class view

You can also check if a user has a particular permission in a view class using the **UserPassesTestMixin** and the **permission_required** decorator.

```
@permission_required('myapp.view_book')
class BookListView(UserPassesTestMixin, ListView):
    model = Book
    template_name = 'book_list.html'
    context_object_name = 'books'

    def test_func(self):
        return self.request.user.has_perm('myapp.view_book')
```

Check login permissions in View

The easiest way to restrict access to your functions is to apply the **@login_required** decorator to your view function

If the **user is logged** in then your view code will execute as normal.

If the user is not logged in, this will redirect to the login URL defined in the project settings (**LOGIN_URL**), passing the current absolute path as the next URL parameter.

If the user succeeds in logging in then they **will be returned back to this page**, but this time authenticated.

In your class-based views derive from **LoginRequiredMixin** to inherit login check function. You need to declare this mixin first in the superclass list, before the main view class.

```
class MyView(LoginRequiredMixin, View):
    login_url = '/login/'
    redirect_field_name = 'redirect_to'
```

Group

A group is a **collection of users** who have similar roles or permissions.

When a permission is granted to a group, **all members** of that group automatically inherit the permission.

```
from django.contrib.auth.models import Group, User

# create a new group
group = Group.objects.create(name='editor')

# add users to the group
user1 = User.objects.get(username='johndoe')
user2 = User.objects.get(username='janedoe')
group.user_set.add(user1, user2)
```

You can assign permissions to the group using Django's admin site or by code.

```
group = Group.objects.get(name='editor')
permission = Permission.objects.get(codename='can_publish_book')
book_ct = ContentType.objects.get_for_model(Book)
group.permissions.add(permission, book_ct)
```

Newspaper improvements

Make this changes to newspaper website:

1. Restrict access to pages different than the home to only logged users.
2. Any logged in user can make changes to any article. Restrict this access so that only the author of an article has this permission.
3. Create a new group called “editor” that have the permission to modify existing articles.

More:

- Add the possibility to add comment to articles. (hint: new model Comment, its form, modify view…). Template are already prepared.

A useful site to explore the class generic built-in in Django: https://ccbv.co.uk/